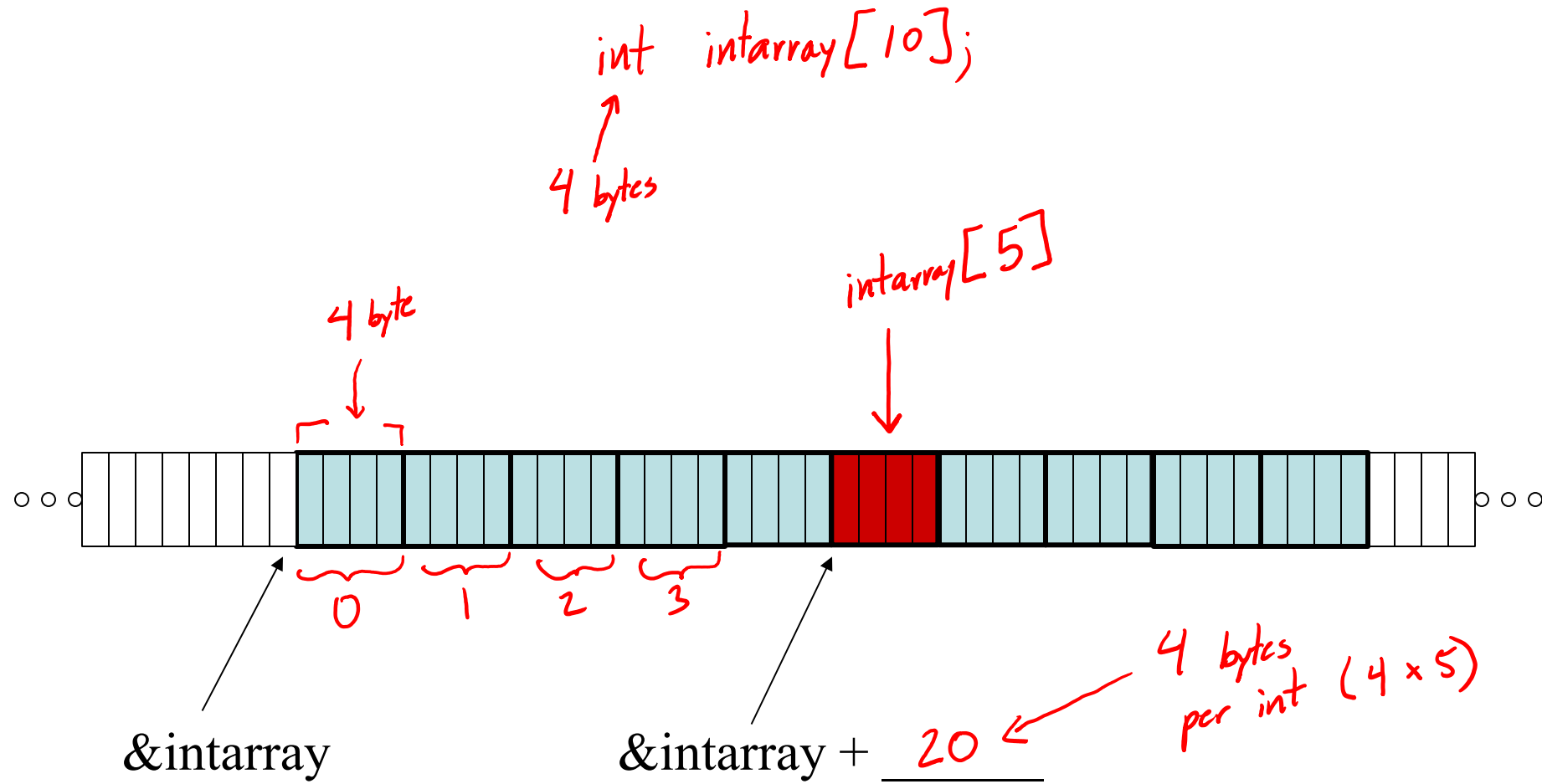


MIPS Registers – 32 registers

Name	Reg Number	Usage	Preserved across call?
\$zero	0	The constant 0	Yes
\$v0-\$v1	2-3	Function results	No
\$a0-\$a3	4-7	Arguments	No
\$t0-\$t7	8-15	Temporaries	No
\$s0-\$s7	16-23	Saved	Yes
\$t8-\$t9	24-25	More temporaries	No
\$gp	28	Global pointer	Yes
\$sp	29	Stack pointer	Yes
\$fp	30	Frame pointer	Yes
\$ra	31	Return address	Yes

Memory Setup in C/Java

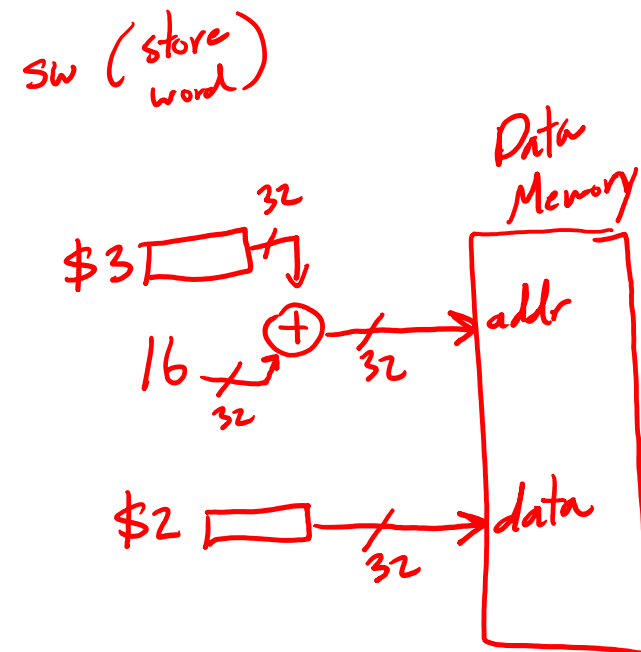
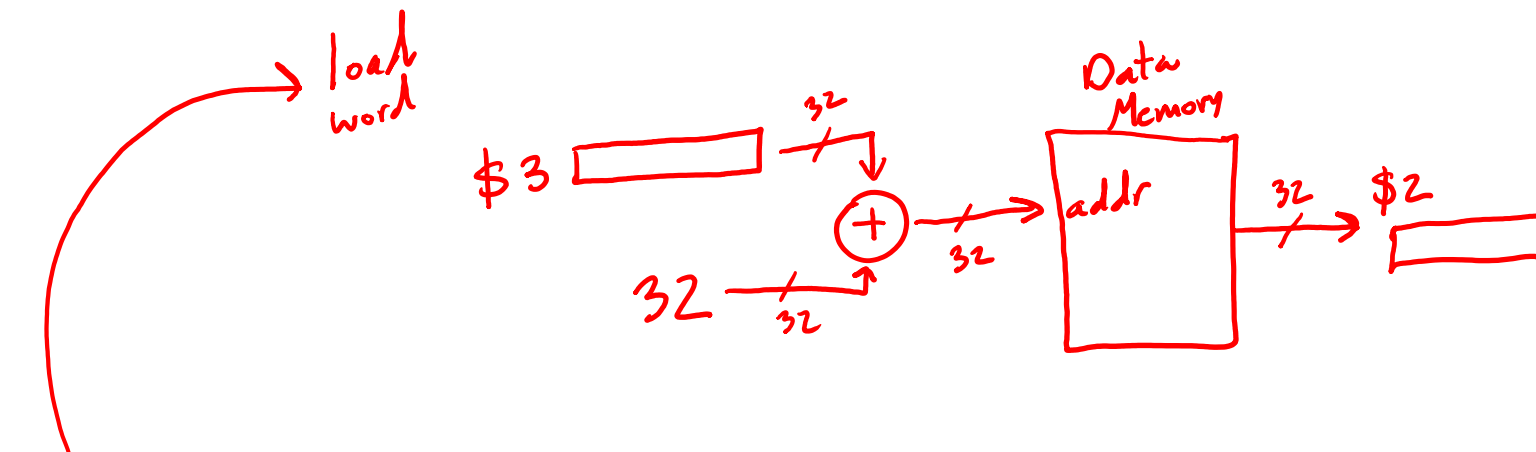


Words and bytes

- 4 bytes = 1 word = 32 bits
- Word addressable — memory is accessed in word-sized chunks
- Byte addressable — each byte can be accessed individually

MIPS is byte addressable

Load/Store Instructions



Operation	rs	rt	rd	shamt	funct	meaning
lw \$2, 32(\$3)	3					\$2 = M[32 + \$3]
sw \$2, 16(\$3)	3					M[16 + \$3] = \$2
lb \$2, 0(\$3)	3					\$2 = M[0 + \$3]
sb \$2, 3(\$3)	3					M[3 + \$3] = \$2

word {

byte {

MIPS Example 2

Translate from C code

```
int A[100];    // ints are 4 bytes in Java/C
char B[100];  // chars are 1 byte in C
```

1. char c = B[50];
2. A[1] = A[5] + 7;

1. 1b \$t1, 50 (\$s1)

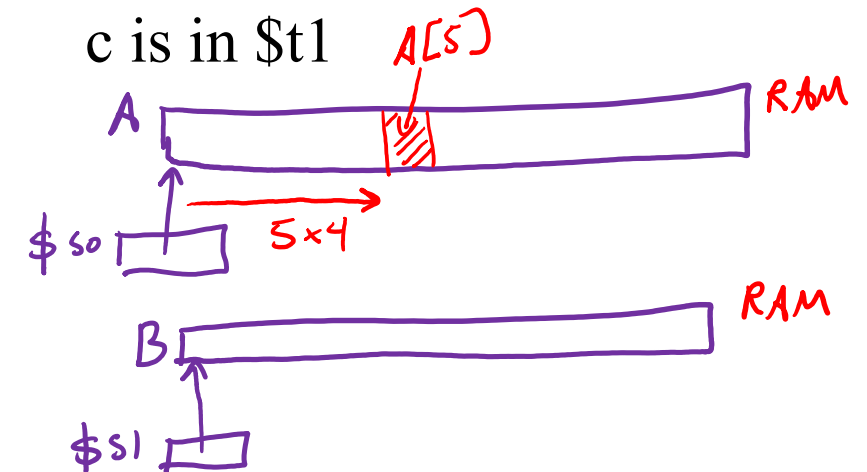
2. lw \$t0, 20(\$s0)
addi \$t0, \$t0, 7
sw \$t0, 4(\$s0)

Assumptions:

&(A[0]) is in \$s0,

`&(B[0])` is in `$s1`

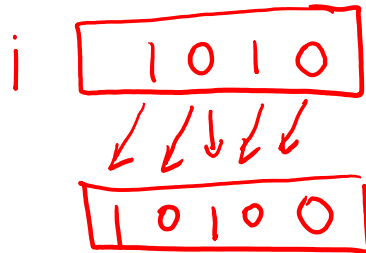
c is in \$t1



MIPS Example 3

Translate:

```
int A[100];  
int i;  
x = A[i];
```



shift left logical
i
start of A
i × 4

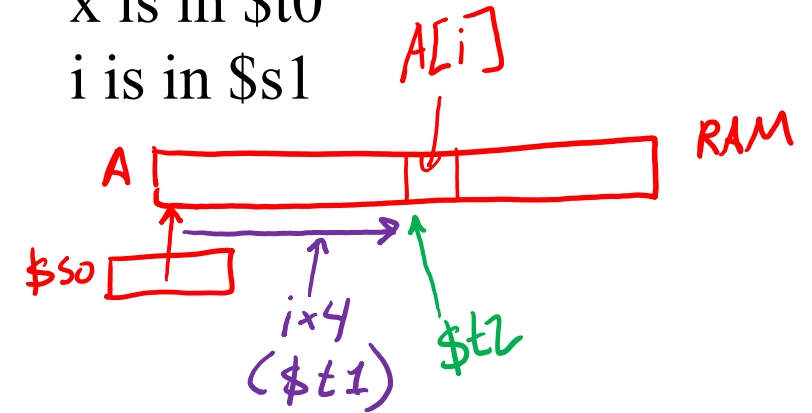
```
sll $t1, $s1, 2  
add $t2, $s0, $t1  
lw $t0, 0($t2)
```

Assumptions:

$\&(A[0])$ is in $\$s0$,

x is in $\$t0$

i is in $\$s1$



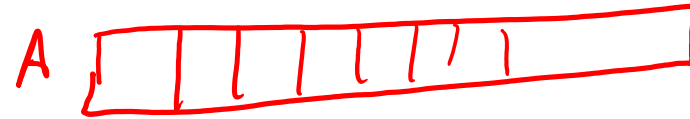
Control Operations

- Unconditional
Branches = jump

- Conditional
Branches = branches

MIPS Example 4

```
sum=0;  
for(i=0; i < 100; i++)  
    sum += A[i];
```



•translated into more detail (with gotos)

```
i = 0;  
sum = 0;  
loop:    if ( i >= 100 )    goto end;  
         sum += A[i];  
         i ++ ;  
         goto loop;  
end:
```


Branches

- goto loop
- if (i >= 100) goto loop

← unconditional

← conditional

jr \$t0

\$t0 200

jump to address
200 in the program
and continue running
from there

Operation	rs	rt	rd	shamt	funct	# comment
beq <i>branch equals</i>	3	2				# if (\$2 == \$3) goto loop
bne	3	2				# if (\$2 != \$3) goto loop
jr <i>jump register</i>		3			8	# goto \$3
j <i>loop</i>						# goto loop